

计算机系统：一个程序的生命之旅



课程核心问题：计算机是如何生成和运行可执行文件的？

本次串讲将以一个程序的视角，回顾整个学期的知识脉络。

旅程的起点：从源代码到字节序列

```
// hello.c
#include <stdio.h>

int main() {
    printf("hello, world\n");
}
```



计算机眼中的 **`hello.c`** 只是一个**字节序列**。所有信息——无论是指令还是数据——都必须**用位(bits)**来表示。

万物皆数：整数的机器级表示

C Code: `short int x = 15213;`

Binary: 00111011 01101101

Hex: 3B 6D

C Code: `short int y = -15213;`

Binary: **1**1000100 10010011

Hex: C4 93

****标题**:** 补码 (Two's Complement)

****内容**:** 最高有效位 (MSB) 为符号位。0代表非负数, 1代表负数。
这是现代计算机表示带符号整数的标准方式。



数据的存放：字节序与对齐

字节序 (Endianness)

Little Endian (x86-64)



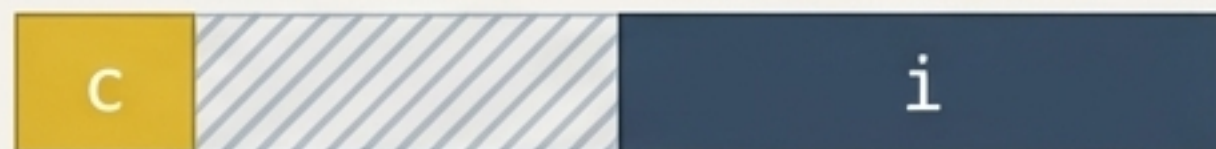
Big Endian (Sparc)



系统内部字节序是一致的，但在跨系统通信时可能引发问题。

对齐 (Alignment)

```
struct S1 { char c; int i; };
```



Total size: 8 bytes

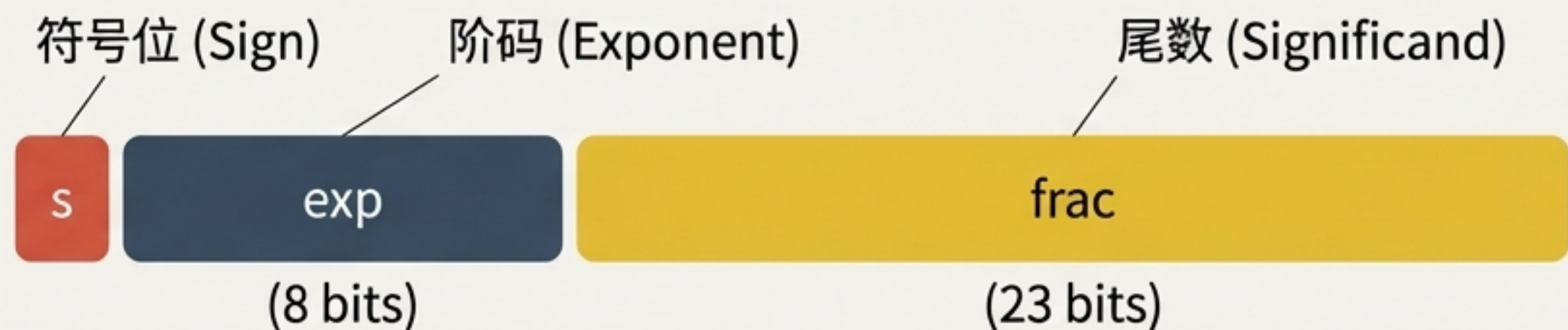
```
struct S2 { int i; char c; };
```



Space saving highlighted

编译器插入间隙以保证字段对齐，提高访存效率。合理的字段顺序可以节省空间。

表示小数与无穷：IEEE 754 浮点数标准



$$V = (-1)^s \times M \times 2^E$$

exp	frac	类型	解释
1-254	任意	规格化数	$E = \text{exp} - 127, M = 1.\text{frac}$
0	$\neq 0$	非规格化数	$E = 1 - 127, M = 0.\text{frac}$
255	0	无穷大 (∞)	表示上溢
255	$\neq 0$	NaN	"Not a Number", 表示无效操作

通过阶码的偏移和尾数的隐含前导1，IEEE 754高效地表示了极大范围的数值。

程序的形态转换之旅



从人类可读的C代码，到机器可执行的0和1，程序经历了一系列精密的转换。

我们将深入其中的关键步骤：编译与链接。

编译：将高级语言翻译为机器指令

C Code
(程序员视角)

```
temp = v[k];  
v[k] = v[k+1];
```



Assembly Code
(编译器/汇编器视角)

```
lw $15, 0($2)  
lw $16, 4($2)  
sw $16, 0($2)  
sw $15, 4($2)
```



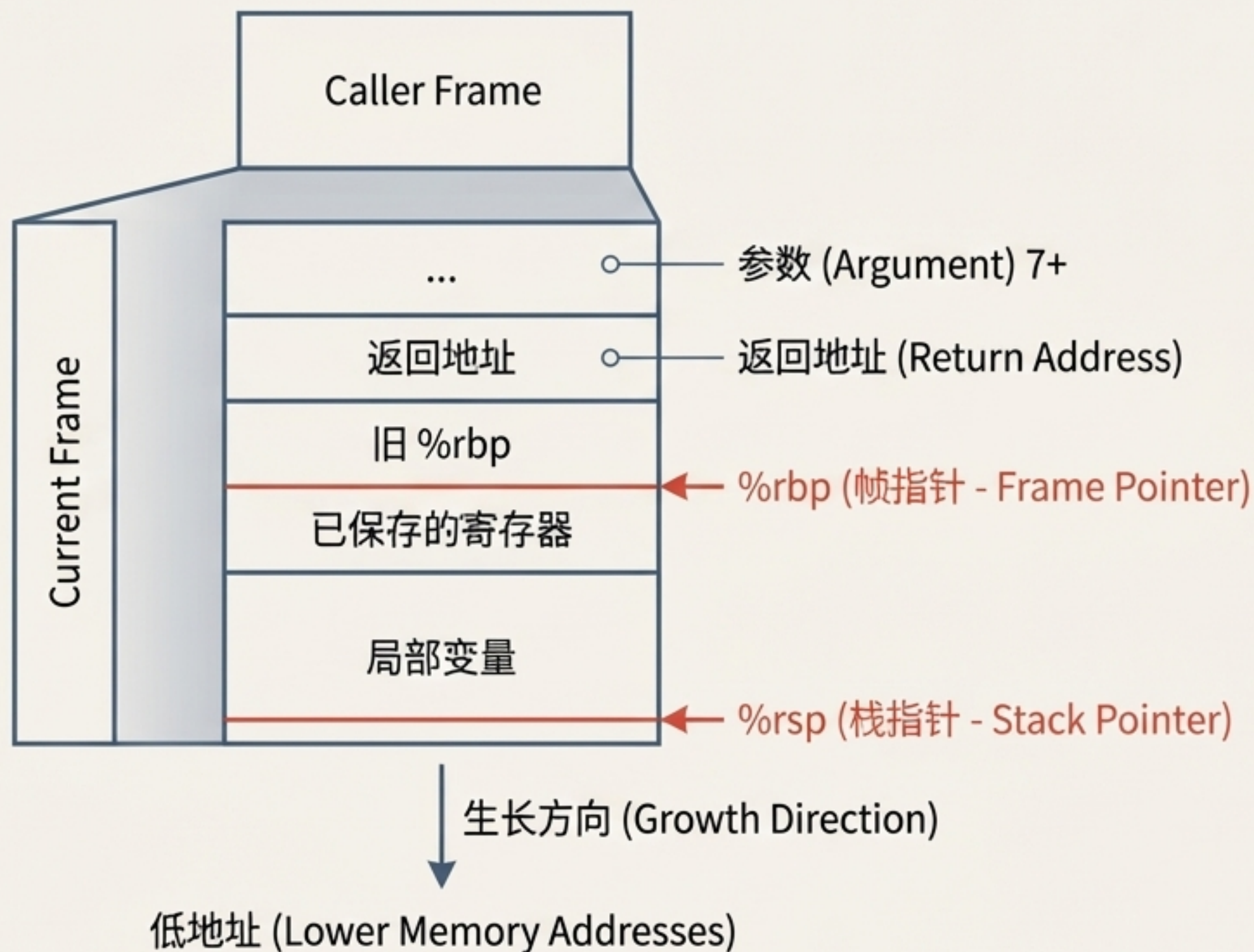
Machine Code
(硬件视角)

```
10001100010011110000...  
10001100010100000000...
```

标题：指令集体系结构 (ISA)

ISA是软硬件之间的契约。它定义了处理器状态（寄存器、内存）、指令格式和效果。编译器的工作就是将C程序转换为遵循ISA的指令序列。

过程调用的核心机制：栈帧



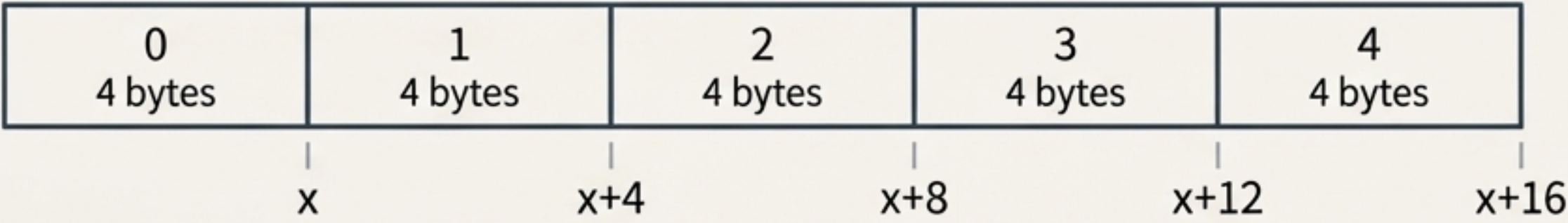
关键机制

1. **传递控制 (Control Transfer)** : ``call`` 指令压入返回地址并跳转。``ret`` 指令弹出地址并返回。
2. **传递数据 (Data Transfer)** :
 - 前6个参数通过寄存器 (``%rdi, %rsi, %rdx, %rcx, %r8, %r9``) 。
 - 返回值通过 ``%rax`` 。
 - 更多参数通过栈传递。
3. **管理内存 (Memory Management)** : 在栈上为局部变量分配和释放空间。

复杂数据在内存中的实现

数组 (Arrays)

C Code:
`int val[5];`

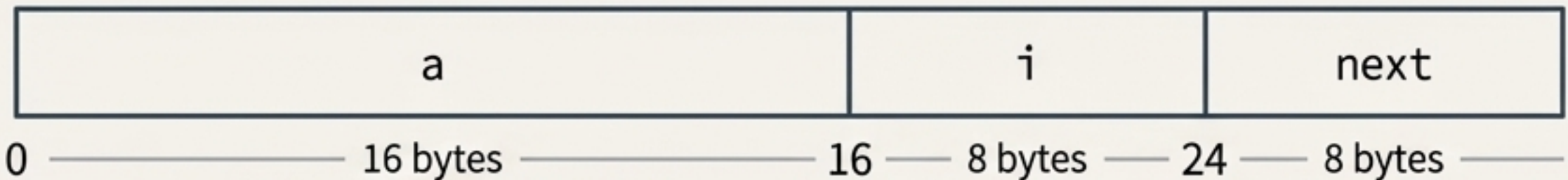


Accessing `val[i]`:
汇编: `movl (%rdi, %rsi, 4), %eax`

地址 = 基地址 (%rdi) + 索引 (%rsi) × 元素大小 (4)。

结构体 (Structs)

C Code:
`struct rec {
 int a[4];
 size_t i;
 struct rec *next;
};`

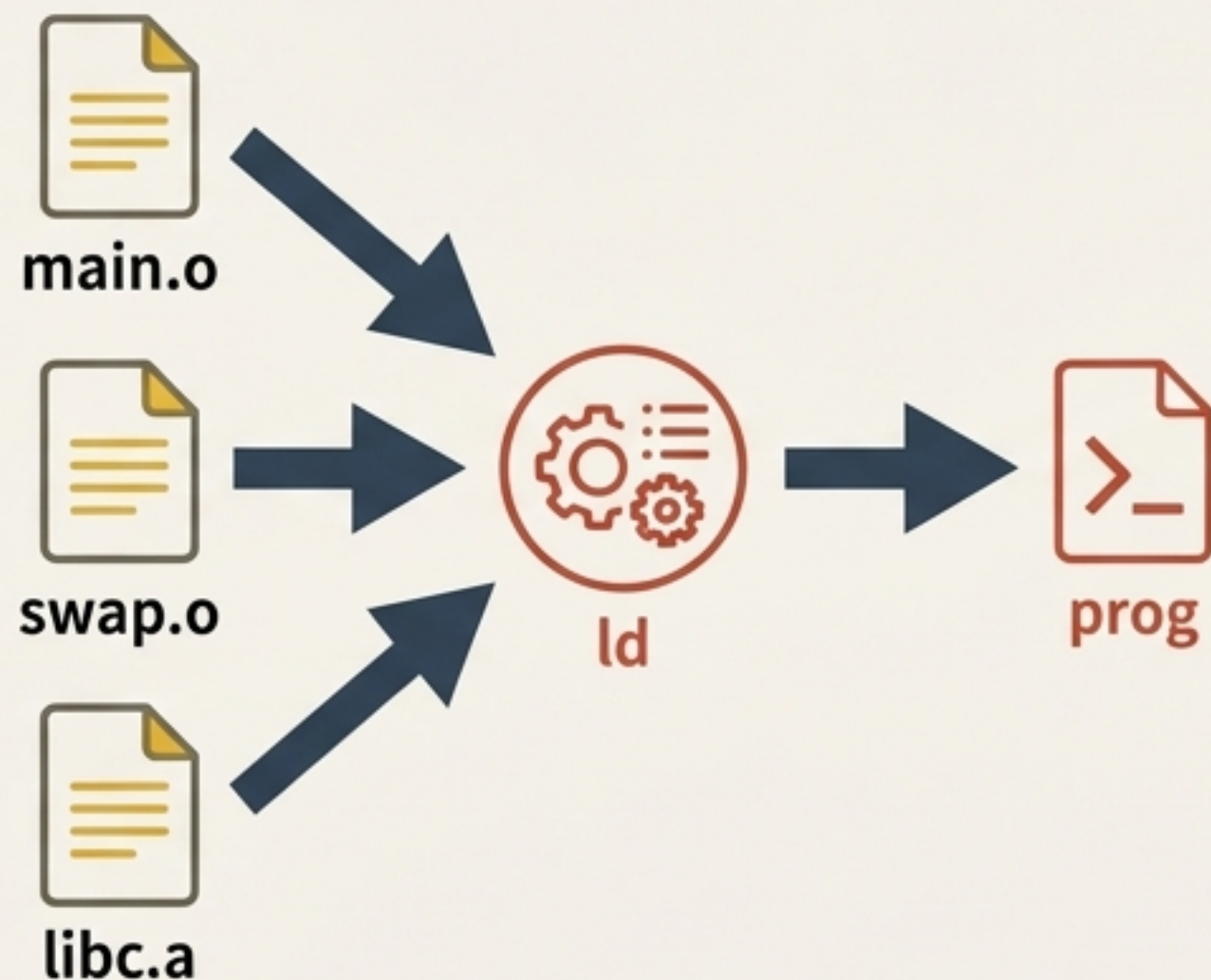


Accessing `&r->a[idx]`:
汇编: `leaq (%rdi, %rsi, 4), %rax`
地址 = 结构体基地址 (%rdi) + 数组偏移 (0) + 索引 (%rsi) × 元素大小 (4)。

汇集百川：链接器的角色

Why Linkers?

- **模块化 (Modularity)** : 将大型程序分解为可独立编译的小模块。
- **效率 (Efficiency)** : 只需重新编译修改过的文件, 然后重新链接。



链接器的两大任务

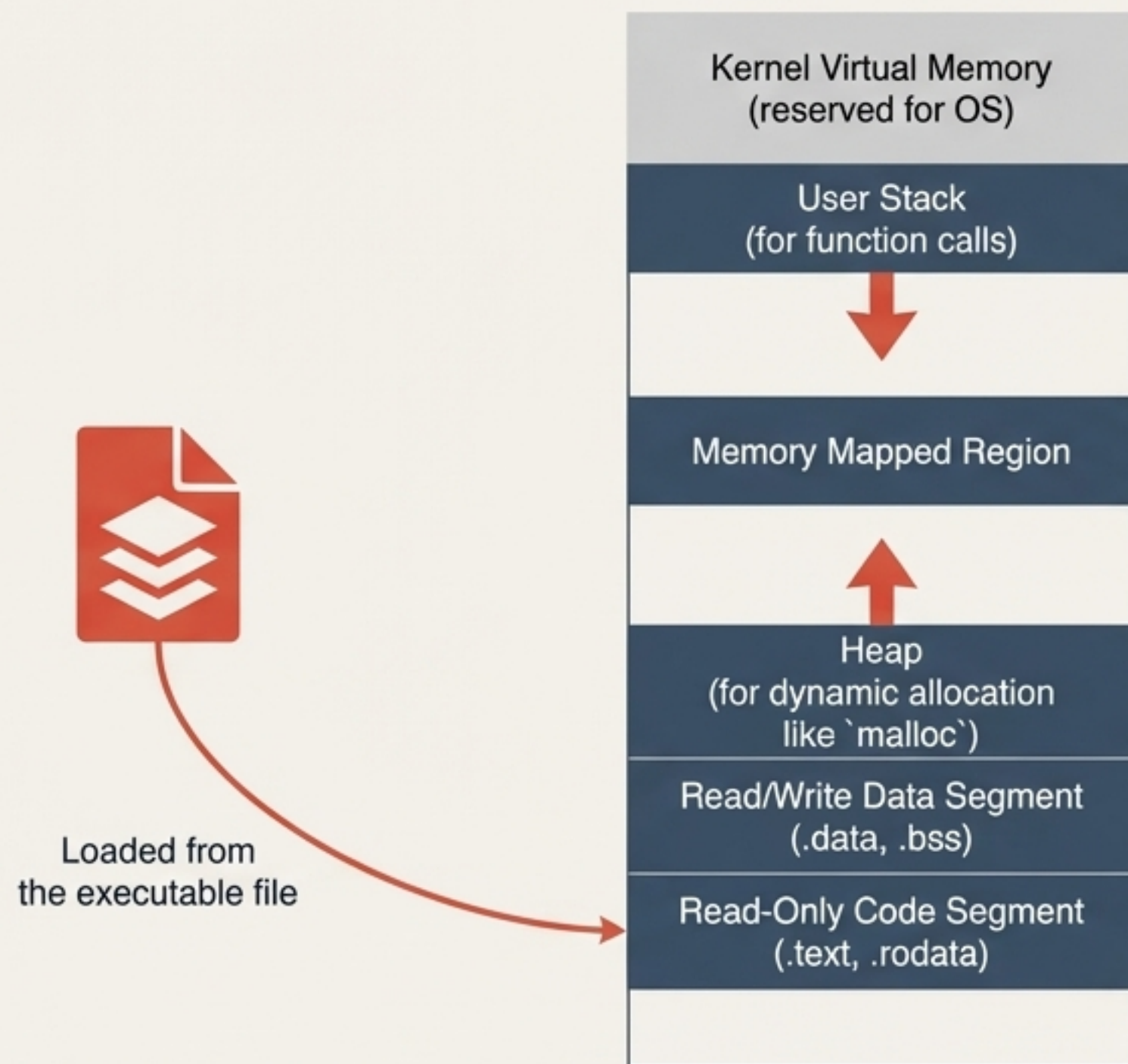
1. **符号解析 (Symbol Resolution)** : 将每个符号引用与一个确定的符号定义关联起来。区分**强符号** (已初始化的全局变量/函数) 和**弱符号** (未初始化的全局变量)。
2. **重定位 (Relocation)** : 合并代码段和数据段。将符号的相对地址修正为最终的绝对虚拟地址。

最终形态：可执行与可链接格式（ELF）



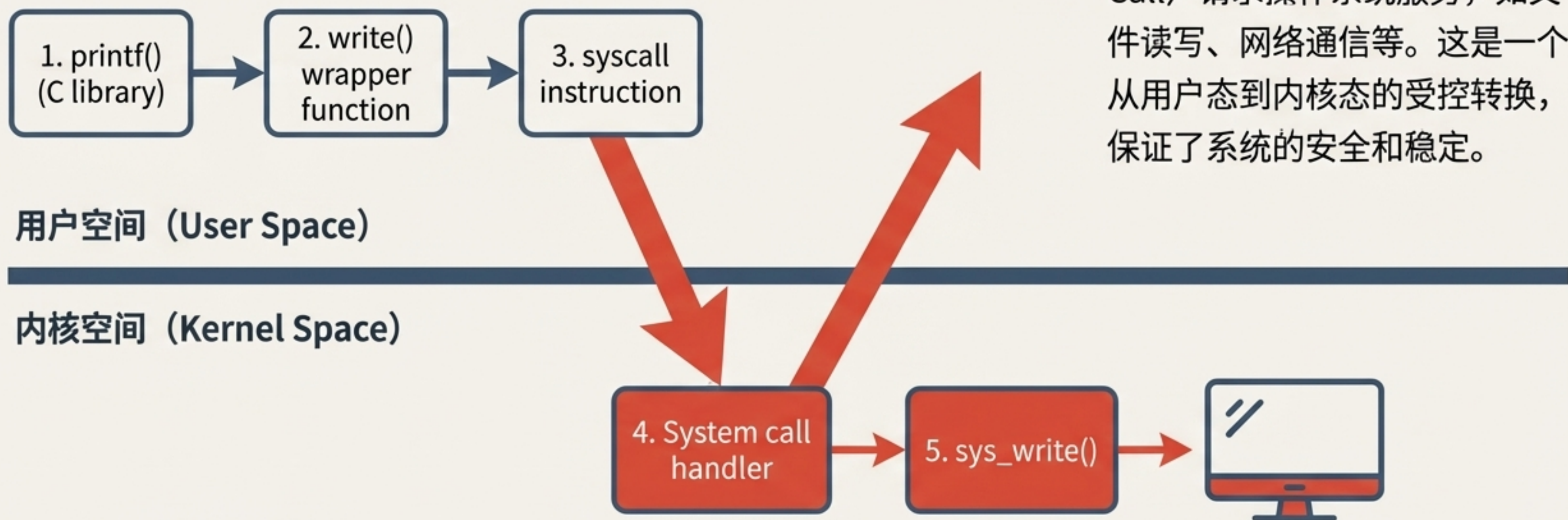
ELF是一种标准格式，统一了可重定位文件(.o)、可执行文件和共享库(.so)的结构。链接器的输出和加载器的输入都是这种格式。

程序的运行：从文件到进程



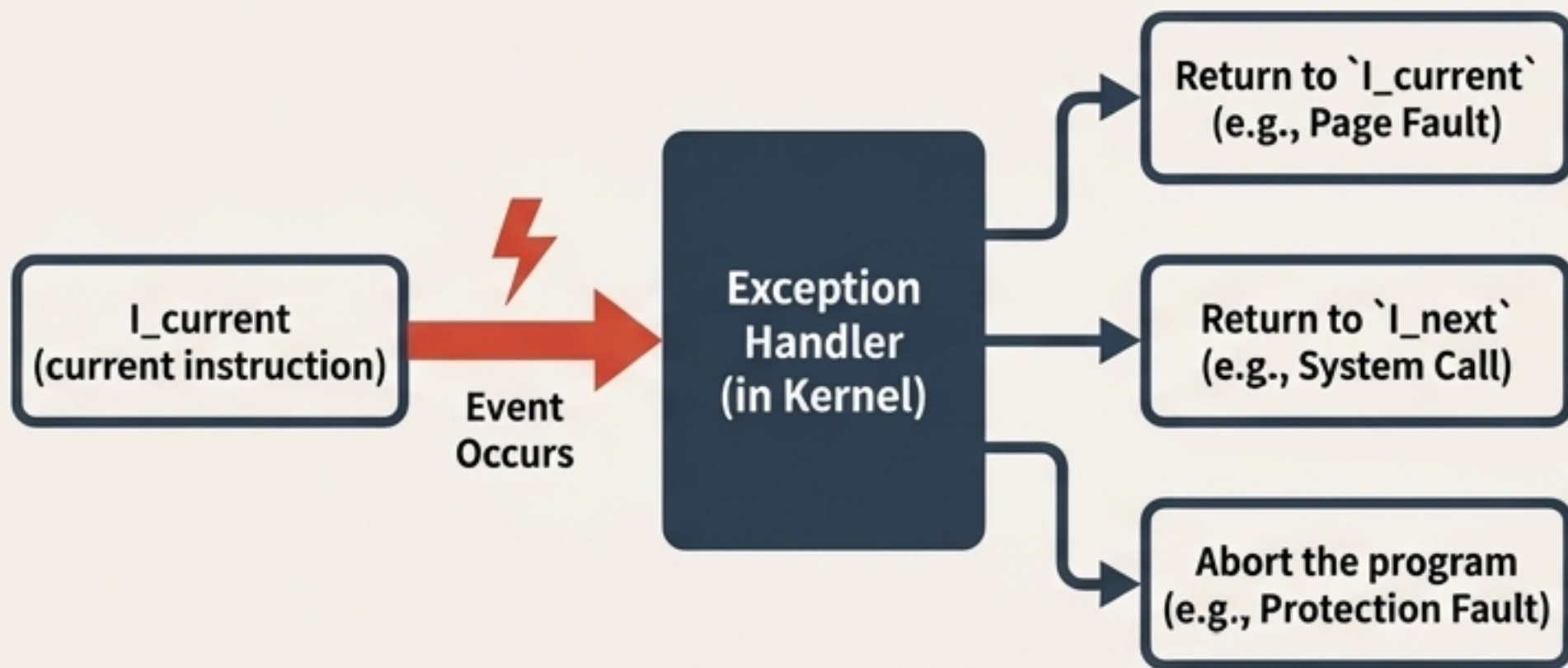
操作系统加载器（`loader`）读取可执行文件的段头表，将代码和数据段映射到进程的虚拟地址空间。程序此时才真正拥有了运行的“场地”。

与世界互动：系统调用与I/O



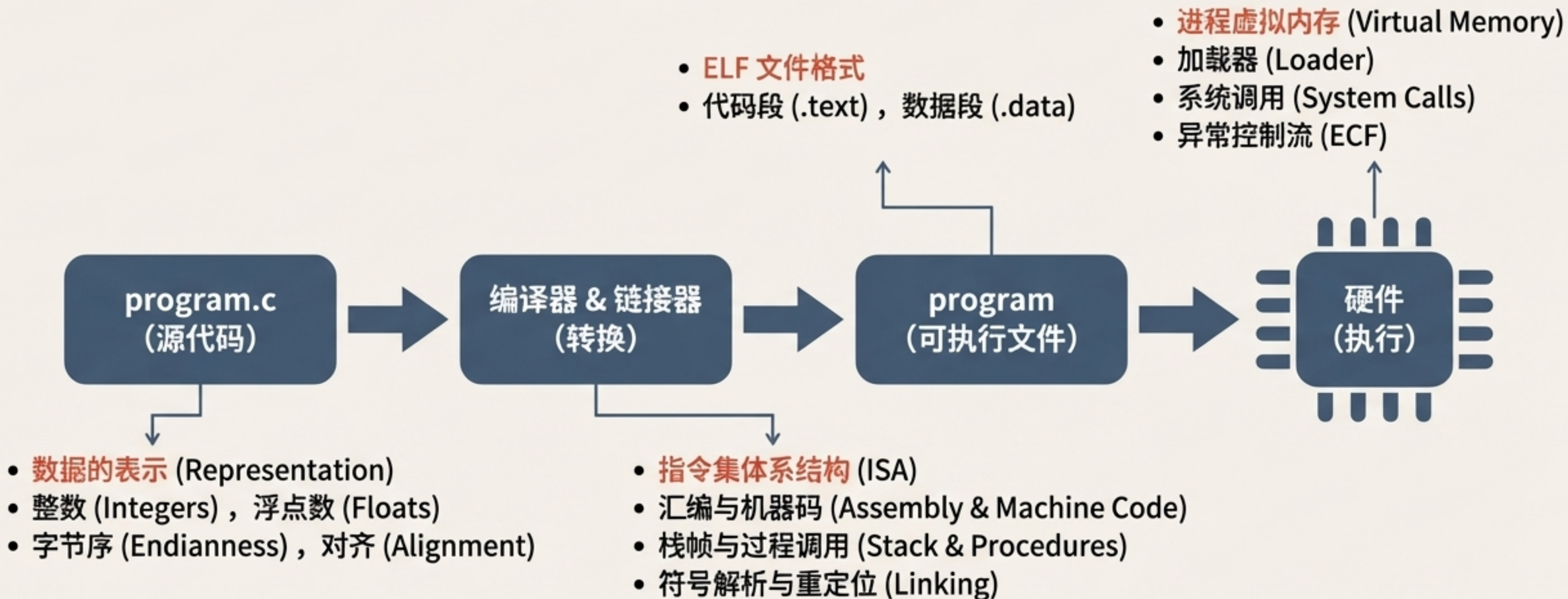
应对突发：异常控制流（ECF）

“异常是为响应处理器状态的某些变化，而将控制权转移到操作系统的事件。”



- 异步（Asynchronous） / 中断（Interrupts）
 - 来自处理器外部的的事件（e.g., I/O完成，定时器）。
- 同步（Synchronous）
 - 陷阱（Traps）：有意的异常（e.g., 系统调用，断点）。
 - 故障（Faults）：可能被修复的错误（e.g., 缺页故障）。
 - 终止（Aborts）：不可恢复的致命错误（e.g., 硬件错误）。

旅程回顾：一个完整的故事



从一个简单的C文件开始，我们跟随它经历了表示、转换和执行的**完整生命周期**。这不仅是程序的旅程，也是整个计算机系统的**核心运作图景**。